

Google Sign-In Integration: Architecture & Implementation Guide

This document maps out exactly how Google Authentication was implemented into our application using NextAuth.js (Auth.js) alongside our existing custom Prisma backend, avoiding any disruption to your current email/password flow.

1. Core Technologies Used

1. NextAuth.js (v4): The most robust authentication library for Next.js, securely handling the OAuth 2.0 handshake, cookie sessions, and encrypted state.
2. @next-auth/prisma-adapter: An adapter that binds NextAuth.js to your existing Prisma ORM setup (ORM setup means configuring a tool that allows us to interact with the database using code objects instead of writing raw SQL queries).
3. Google OAuth 2.0 Provider: The service linking your Next.js application to Google's Identity services (When a new user logs in via Google, NextAuth receives their email from Google's OAuth response, so we directly use that email to create the user in our database).

2. Implementation flow

A. Database Schema (prisma/schema.prisma)

NextAuth requires specific tables to map Google profiles securely to your application. We updated your schema to append:

Account: Stores external OAuth mappings (like providerAccountId and OAuth tokens).

Session & VerificationToken: Standard NextAuth boilerplate tables required by the Prisma Adapter.

User enhancements: Modified the User table slightly to fit NextAuth's strict types (changing emailVerified from Boolean to DateTime? and appending image). Also linked relational arrays for sessions and accounts.

B. Core NextAuth Endpoint (app/api/auth/[...nextauth]/[route.ts](#))

Instead of letting NextAuth automatically insert raw Users into our database blindly (which would crash because NextAuth uses String IDs while your DB uses Integers), we built a Custom Prisma Adapter Wrapper.

Handle Generation: Safely derives unique Albiz handles (e.g. @johndoe) derived automatically from their Google emails.

ID Mapping: Safely calculates the `.aggregate({ _max: { id: true } })` logic to attach the next sequential Integer ID.

C. Client-Side State Synchronization (app/(main)/layout.tsx)

Because our application heavily relies on an existing AuthContext to determine things like userRole ('CIRCLE' or 'NORMAL') and currentUserId, NextAuth's isolated session system needed bridging.

SessionProvider: Wraps your application to expose NextAuth state.

AuthSyncWrapper: A custom React component that actively listens. If it detects a logged-in NextAuth Google session, it extracts their role and automatically calls `AuthContext.signIn(role, id)`, dynamically converting a Google user into your app's native authorized state.

Google Logic: Swapped out the blank `onClick={() => {}}` logic previously on your buttons with NextAuth's `signIn("google")`.

3. What You Need to Setup (Google Cloud Console)

For this integration to actively work on a live domain (and efficiently locally), you must ensure your Google Cloud Setup is correct.

Start
↓
Go to Google Cloud Console
↓
Create / Select Project
↓
Go to "APIs & Services"
↓
Open "Credentials"
↓
Create "OAuth Client ID"
↓

Select Application Type:

→ Web Application



Add Authorized JavaScript Origins

→ http://localhost:3000

→ https://your-production-domain.com



Add Authorized Redirect URIs

→ http://localhost:3000/api/auth/callback/google

→ https://your-production-domain.com/api/auth/callback/google



Click Create



Get Client ID & Client Secret



Add to .env file



Integration Ready

The raw credentials provided directly from the Google Cloud Console

GOOGLE_CLIENT_ID="81914..."

GOOGLE_CLIENT_SECRET="GOCSPX-..."

The Base URL of your app (change this in production)

NEXTAUTH_URL="http://localhost:3000"

A randomized strong string used to encrypt JWT session tokens in cookies

NEXTAUTH_SECRET="super_secret_albiz_auth_key_12345"